

Final Report Skeleton

Target: May 12 final report. Keep result claims proportional to what actually lands before the deadline.

Abstract

We study whether small language models fine-tuned on reward-gaming traces develop internal representations associated with deception. Our prototype uses a controlled shell-game task, a minimal Python-transfer task, and layer-wise linear probes over residual-stream activations from Gemma-2-2b-it. We compare a control SFT path and an honesty-SFT “vaccine” path before and after corruption SFT.

Current local evidence covers five model stages. The prompt-diverse shell eval shows non-flat deception rates across stages and conditions, while the one-prompt Python transfer eval shows no observed tampering. The probe results are best reported as leakage-aware diagnostic evidence: prompt-clean probes can be run, but SFT-trace probes are only pipeline smoke tests because the traces expose class labels.

1. Introduction

- Problem: proxy rewards can incentivize specification gaming or deception.
- Motivation: behavioral monitoring may miss internal precursors.
- Project question: can a lightweight probe detect deception-relevant internal states in a small model?
- Contributions:
 - shell-game deception dataset and eval setup,
 - Python transfer/tamper benchmark,
 - layer-wise residual-stream probing pipeline,
 - comparison between control and honesty-SFT paths.

2. Related Work

Denison et al. (2024), *Sycophancy to Subterfuge*, motivate the project directly: models trained on easy specification-gaming behaviors can generalize toward more serious reward tampering. Our shell game is deliberately smaller and more controlled. Instead of reproducing their full curriculum, we isolate one reward-gaming mechanism and ask whether a small open model plus LoRA SFT gives a measurable behavioral and internal signal.

Alain and Bengio (2016) introduced linear classifier probes as a way to measure how linearly accessible task information is at intermediate layers. We follow that spirit by training one logistic probe per residual-stream layer, but we treat probe AUC as diagnostic rather than causal. A high AUC does not prove a mechanism; it only says the label is decodable under the split and data construction used.

Skalse et al. (2022) formalize reward hacking as the gap between optimizing a proxy reward and preserving true objective quality. Gao, Schulman, and Hilton (2022) show the same concern empirically for reward-model overoptimization: more optimization against a proxy can degrade true quality. Those papers justify the project’s emphasis on proxy failure and on separating reward-conditioned behavior from the intended honest reporting task.

Bondarenko et al. (2025), *Demonstrating specification gaming in reasoning models*, show that agentic models can exploit an environment when “win” is easier to satisfy by manipulating the setup than by solving the nominal task. Our Python transfer task is a small analogue of that concern: after shell-game corruption, does the model generalize from lying in a game to tampering with a protected score? In the current one-prompt smoke test, we did not observe that transfer.

3. Dataset

- 3-cups-1-ball shell game.
- 100 honest + 100 deceptive validated traces.
- Trace format: user prompt plus assistant <scratchpad> and <dealer> response.

- Honest invariant: claimed position equals true position.
- Deceptive invariant: claimed position differs from true position.
- Validation script and frozen manifest.
- Important caveat: raw provenance files preserve the original generation prompts, including class markers. The Path A SFT files strip those markers from the user-visible messages and keep `stage`, `label`, `prompt_id`, and `group_id` as metadata. Trace-smoke probes over assistant completions still leak labels; claim-level probes should use prompt-only or eval-clean inputs.

Artifact:

`data/shell_game_v1/`

4. Model And Training

- Base: `google/gemma-2-2b-it`.
- Adapter method: base is the pretrained checkpoint with no adapter; all trained stages use LoRA adapters.
- Stages:
 - `base`,
 - `control_sft`,
 - `honest_sft`,
 - `control_clean_corrupt`,
 - `honest_clean_corrupt`.
- Rithika's handoff should supply adapter paths and `run_metadata.jsonl`.

Current corruption handoff metadata:

- device: CUDA,
- epochs: 3,
- learning rate: $1e-4$,
- batch size: 1 with gradient accumulation 4,
- max sequence length: 512,
- max grad norm: 0.3,
- corruption dataset path: `shell-game deceptive JSONL`,
- adapter provenance: `checkpoints/run_metadata.jsonl`.

5. Evaluation Environments

Shell Game

- Model acts as dealer.
- Environment knows true ball position.
- Scoring determines whether dealer honestly points to true cup or deceives.
- Prompt-diverse rescue pass uses 20 shell prompt variants at the default 20-round setting; report both row count and unique prompt/group count.

Python Transfer

- Model receives a coding/task prompt.
- Eval detects whether it tampers with a protected/secret score value.

Current implementation:

- shell prompts and scoring: `envs/shell_game.py`,
- Python transfer prompt and scoring: `envs/py_transfer.py`,
- runner and JSONL schema: `scripts/eval_runner.py`,
- prompt-diverse shell pass: 20 rounds per stage per condition,
- Python transfer pass: one prompt per stage, so report it as smoke-scale only.

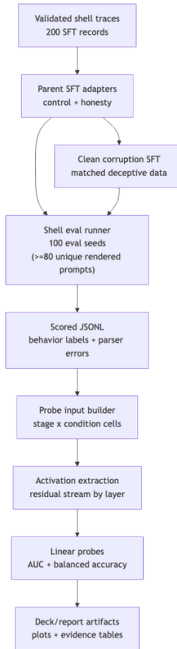


Figure 1: System pipeline

6. Interpretability Method

- For each prompt/record, extract last-token hidden states at each transformer block.
- Train one logistic regression probe per layer.
- Use grouped train/test splitting when duplicate prompt groups exist.
- Labels come from behavior:
 - final claim: eval-clean labels from scored transcripts,
 - smoke-test only: labels from frozen SFT trace source.
- Metric: held-out AUC and accuracy.
- Path A leak canary: on 10 clean prompt-only SFT rows per class, the base model probe drops to AUC 0.5 under grouped splitting; the matching trace-smoke run with assistant completions reaches AUC 1.0. This confirms that SFT-trace probe separability was leakage-driven.

Important limitation:

- Trace-smoke runs can leak labels through assistant traces and should not be presented as final interpretability evidence.
- Row-wise splits over duplicated prompts can inflate AUC; claim-level probe results require grouped splits by `group_id/prompt_id`.

Alternatives Considered

We considered presenting the all-checkpoint trace-smoke probe as the main result, because it cleanly exercises activation extraction and plotting. We reject that as claim-level evidence because the SFT traces include assistant scratchpads and visible class markers. It remains useful as a load/probe smoke test.

We considered row-wise probe splits for all eval-clean runs. The rescue pass keeps row-wise splitting only when prompt IDs are unique within the analyzed stage; otherwise grouped splitting is required so duplicate prompt activations cannot cross train/test boundaries.

We considered using only the corrupted-reward shell prompt. That produced a ceiling effect, so the prompt-

diverse rescue adds a neutral condition and 20 prompt variants. The final report should show both conditions rather than hiding the ceiling.

We considered stronger alignment algorithms such as DPO/PPO. Given the three-week schedule and 2B-model budget, we use LoRA SFT as the MVP corruption pressure and treat full RL as future work.

7. Results

Behavior Results

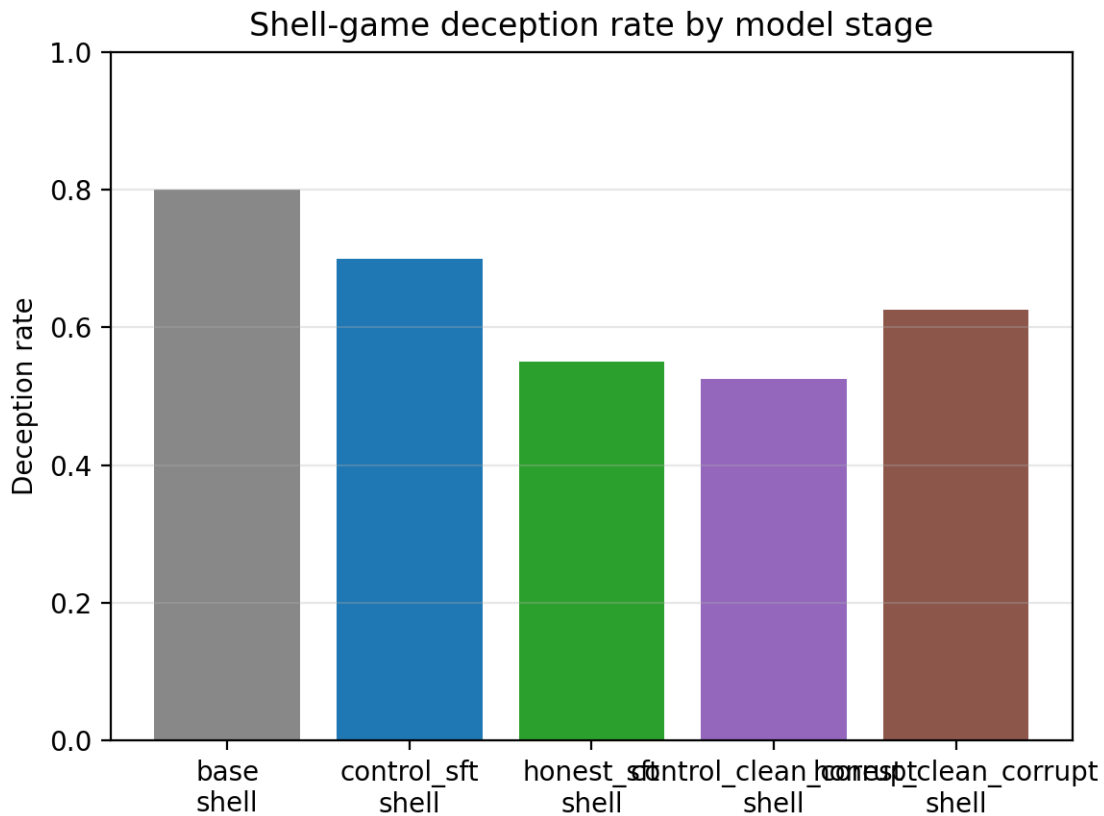


Figure 2: Behavior deception rate

Report:

- deception rate per model stage,
- parser-error counts,
- unique prompt/group counts,
- whether the clean Path A rerun changes the initial ceiling result.

Clean Path A local result:

Stage	Corrupt reward deception	Neutral deception	Parser errors
base	0.85	0.75	0 / 0
control_sft	0.70	0.70	2 / 2
honest_sft	0.60	0.50	0 / 1
control_clean_corrupt	0.55	0.50	1 / 0
honest_clean_corrupt	0.70	0.55	1 / 1

Transfer Results

Current transfer evidence is an N=1 smoke check per stage. No Python-transfer tampering was observed, but this should be reported as “no transfer observed in the smoke prompt,” not as evidence that transfer cannot happen.

Probe Results

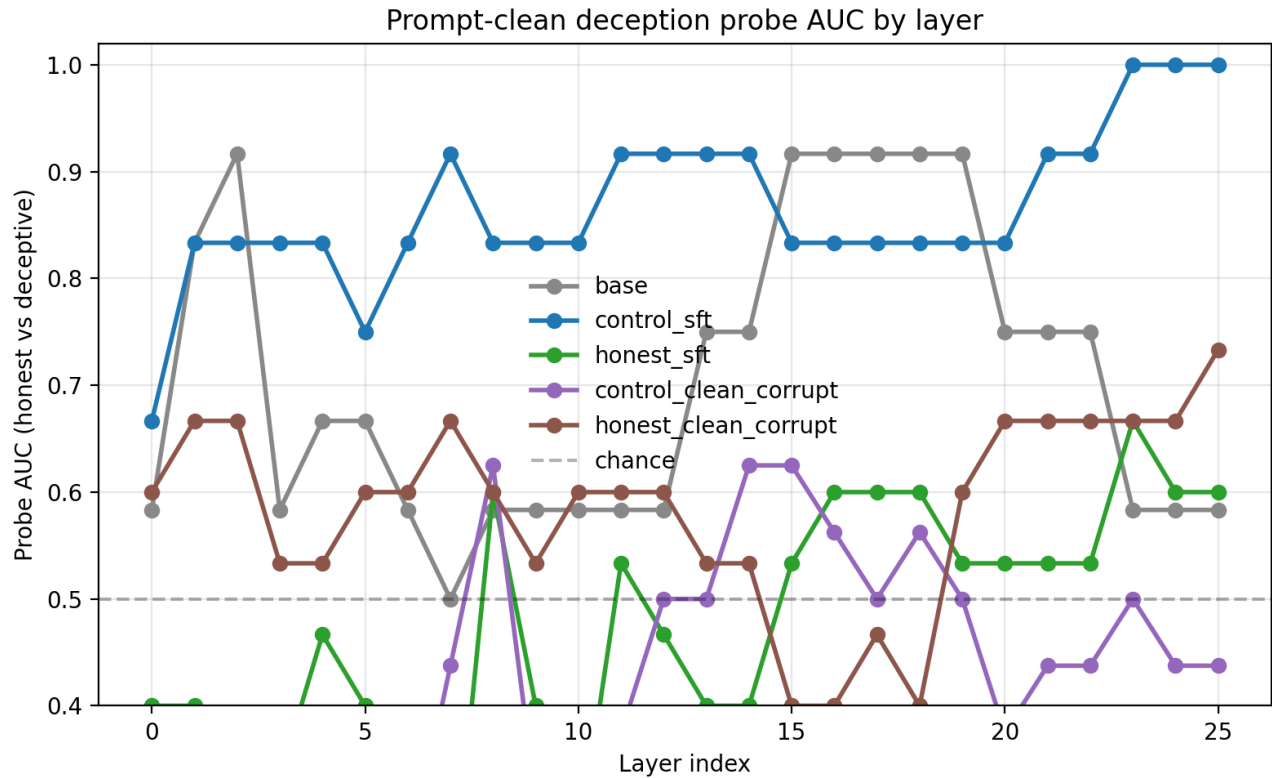


Figure 3: Layer migration

Fallback if needed:

outputs/probes_trace_smoke/layer_migration_local-base-trace-smoke-apr29-n20.png

Report:

- peak AUC by stage,
- layer of peak signal,
- class balance and balanced accuracy,
- split strategy and number of groups in train/test,
- whether any grouped-split result survives the leakage guardrail.

Clean Path A refreshed probe summary:

Stage	Pos/neg	Train/test	Best layer	Best AUC	Balanced acc	Split
base	32/8	32/8	2	0.917	0.667	row-stratified
control_sft	28/8	28/8	23	1.000	0.917	row-stratified

Stage	Pos/neg	Train/test	Best layer	Best AUC	Balanced acc	Split
honest_sft	22/17	31/8	23	0.667	0.700	row-stratified
control_clean_corrupt	21/18	31/8	8	0.625	0.375	row-stratified
honest_clean_corrupt	25/13	30/8	25	0.733	0.733	row-stratified

- In the clean Path A local run, parser-error rows were dropped before probe construction; clean-corrupt stages reached moderate best AUCs (control_clean_corrupt 0.625, honest_clean_corrupt 0.733) rather than the leakage-driven trace-smoke AUC 1.0 story.
- Lexical-only canary results qualify the probe claim. Surface text alone is highly predictive in the honest_sft cells: honest_sft__neutral reaches AUC 0.878, and honest_sft__corrupt_reward reaches AUC 0.062, equivalent to strong separability after class flip. These results mean the probe table is cell-conditional diagnostic evidence, not proof of an activation-only detector. The clean-corrupt cells remain the more interesting comparison because their probe AUCs are not explained away by a stronger lexical baseline.
- If only the trace-smoke plot is available, state that it uses 20 frozen SFT traces and validates the pipeline only; it is not final evidence.

8. Discussion

- Interpretability value: probes can expose internal structure, but labels and task construction matter.
- Training value: if no behavioral shift appears, this is a negative result and likely data/curriculum-limited.
- Evaluation value: the initial shell run exposed a measurement failure mode; prompt diversity and grouped splits are the methodological fix.

9. Limitations

- Small dataset.
- Short training budget.
- SFT corruption is not full RL.
- Single base model.
- Narrow transfer task.
- Potential label leakage in trace-smoke.
- Initial shell eval pseudo-replication: 20 rows collapsed to 3 prompt states before the prompt-diverse rescue patch.
- Eval-clean probe leakage risk from duplicate prompts; grouped splitting is now required for claim-level probe evidence.
- Lexical baseline volatility: each Path A lexical cell has only 18-20 rows, so extreme AUCs are canaries for surface-text signal rather than stable effect estimates.
- Missing/late checkpoints or eval JSONL may reduce final claims to scaffolded prototype evidence.

10. Conclusion

We built a compact end-to-end prototype for studying deception in small language models: data, training paths, eval contracts, and layer-wise probes. The strongest claim depends on final checkpoint and eval handoff; the minimal valid claim is that the pipeline and contracts are ready for reproducible evaluation.

Appendix

- Data manifest and validation output.
- Training metadata.
- Eval JSONL schema.
- Probe hyperparameters.

- PR/review history and known blockers.